

DynSTEER: Dynamic Stage-wise Trajectory Evaluation and Execution-time Review for Agents

Anonymous authors
Paper under double-blind review

Abstract

Large language model agents are increasingly deployed for long-horizon task execution, raising a central granularity question for trajectory evaluation: whole-trajectory verification is too coarse to capture concrete failures and their associated evidence in long trajectories, while atomic-step scoring is too fine-grained, noise-sensitive, and computationally expensive. This granularity gap makes a single-reference trajectory paradigm inadequate for assessing the rich space of valid agent execution paths and delays timely feedback and early stopping in long-horizon tasks. To address these issues, we propose DynSTEER, a dynamic stage-wise framework for agent trajectory evaluation. DynSTEER bridges the granularity gap through stage-wise dynamic evaluation that segments rollouts at key execution nodes and adapts its multi-level review strategy based on stage-level results; it compiles a path-tolerant milestone graph from available task inputs to preserve diverse legal paths without reference leakage; and it supports terminating unrecoverable agent executions to curb resource waste. Across ToolSandbox and SWE-bench Pro, DynSTEER improves evaluation discriminability by 85.2% and 164.0% over native evaluation and saves 17.74% and 8.40% of execution steps across paired trajectories.

1 Introduction

LLM agents are increasingly undertaking complex, long-horizon tasks across real-world environments (Yao et al., 2023; Shinn et al., 2023; Jimenez et al., 2024; Gu et al., 2024; Zhou et al., 2024). As task horizons expand, evaluating only final output results becomes insufficient to identify agent deficiencies and diagnose failure modes, therefore bringing trajectory-level evaluation to the forefront of agent research (Qian et al., 2024; Shi et al., 2026).

However, existing trajectory evaluation paradigms face a central granularity problem, especially for long-horizon tasks. Whole-trajectory verification is too coarse to focus on concrete failures and their associated evidence in long trajectories, whereas atomic-step scoring is too fine-grained: isolated messages lack task context, so ordinary retries and harmless observations become noise. This granularity gap further raises two challenges. Agent executions often admit multiple valid and effective paths. Because of this granularity gap, evaluation typically offers only limited path references and therefore struggles to faithfully reflect the quality of diverse agent executions. Moreover, full-trajectory review delays diagnosis that could be made earlier, so an execution that has already gone wrong may continue, wasting model resources and triggering additional skill or tool calls.

As illustrated in Figure 1, we introduce DynSTEER, a Dynamic Stage-wise Trajectory Evaluation and Execution-time Review framework. To address the granularity challenge, DynSTEER performs stage-wise dynamic evaluation: it segments rollouts at necessary execution nodes, preserves task context while isolating localized evidence for error attribution, and adapts its multi-level review strategy based on stage-level results. To avoid single-reference bias, it compiles a path-tolerant milestone graph from available task inputs via multi-candidate simulated planning and majority consensus, capturing necessary task invariants while accommodating diverse legitimate paths. Finally, it supports detecting unrecoverable prefixes and terminating them to curb resource waste.

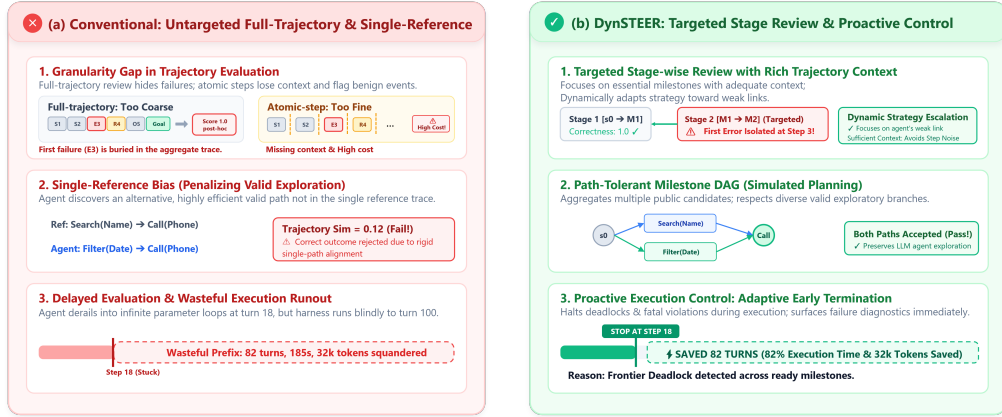


Figure 1: Comparison of Agent Evaluation Paradigms. (a) Conventional paradigms evaluate either coarse outcomes or noisy atomic steps, often against a single reference, and discover waste only after execution. (b) DynSTEER evaluates path-tolerant milestone-bounded stages and uses the resulting stage evidence to target review and stop hopeless prefixes early.

Experiments on ToolSandbox and SWE-bench Pro show that DynSTEER improves discriminability by 85.2% and 164.0% over native evaluation, enhances diagnostic precision, preserves the validity of diverse execution paths, and saves 17.74% and 8.40% of execution steps across paired trajectories.

In summary, our contributions are as follows:

- **Stage-wise Dynamic Evaluation:** We segment rollouts at necessary execution-step nodes and adapt multi-level review routing from stage-level results, isolating context-rich evidence windows for multidimensional scoring and first-error localization.
- **Path-Tolerant Task Blueprint Compilation:** We compile a milestone DAG from available task inputs via multi-candidate planning and strict-majority consensus, preserving diverse valid paths without reference leakage.
- **Counterfactual Early Termination:** We detect unrecoverable prefixes from stage-level evidence and terminate them, substantially reducing wasted computation.

2 Related Work

2.1 Agent Trajectory Evaluation

Unlike outcome benchmarks that inspect only terminal code patches or database records (Jimenez et al., 2024; Qin et al., 2024; Liu et al., 2024; Zhou et al., 2024), trajectory evaluation examines intermediate planning decisions, tool invocations, and error recoveries generated over the course of agent execution. Recent benchmarks expand assessment: BFCL (Patil et al., 2025) evaluates multi-step tool calls via AST checks; TRAJECT-Bench (Li et al., 2025) assesses tool selection and dependency order; AgentRewardBench (Zhou et al., 2025) benchmarks trajectory judges; and DeepRed (Kuo et al., 2026) and Claw-Eval (Zhang et al., 2026) leverage environment snapshots to award partial credit along execution traces.

Despite providing intermediate visibility, existing evaluators face two key limitations. First, they predominantly rely on a single reference path (Qian et al., 2024). In open environments, tasks frequently admit multiple valid solution routes; single-path matching inherently penalizes legitimate exploratory strategies. Second, they operate post-hoc across the full rollout and lack dynamic intervention to adapt evaluation strategies to an agent’s specific weak links.

2.2 Dynamic Evaluation for Agents

Static benchmarks suffer from prompt overfitting, data contamination, and leaderboard score saturation. To address these vulnerabilities, researchers have begun exploring dynamic evaluation paradigms. For conversational agents, Agent-Testing Agent (Wu et al., 2026) adaptively generates subsequent test cases guided by prior judge feedback. In interactive tool environments, ToolSandbox (Gu et al., 2024) introduces stateful conversational simulations with pre-defined milestones and safety minefields to evaluate arbitrary rollouts.

However, existing dynamic evaluation approaches primarily concentrate on dynamically selecting or perturbing benchmark test suites (Gu et al., 2024; Wu et al., 2026). During the actual execution of an individual task, trajectory evaluation remains largely passive: evaluators typically check simple milestone reachability at the end of rollouts, lacking a formal mechanism to dynamically steer evaluation focus, update dimension weights, or control execution progression in real time.

2.3 Efficient Evaluation and Early Stopping Strategies

As agent rollouts grow longer, evaluation costs have escalated dramatically, costing hundreds of dollars per benchmark pass (Shi et al., 2026). Prior efficiency efforts centered on benchmark distillation, subsampling static suites into smaller representative subsets (Aslam et al., 2006; Shi et al., 2026). To improve within-task efficiency, EarlyEval (Shi et al., 2026) introduces early outcome prediction, training dual LightGBM classifiers on prefix features to forecast binary task success/failure and halt execution at an early stage.

Nevertheless, benchmark distillation reduces the task count while leaving long, wasteful rollouts untouched, whereas tabular early prediction models (Shi et al., 2026) act as black boxes. They predict binary outcomes without understanding domain semantics, cannot provide causal failure attribution, and lack awareness of tool dependency constraints or hazardous minefields.

In summary, existing trajectory evaluators either inspect isolated atomic steps that suffer from local noise, or evaluate full traces post-hoc against a single reference path that penalizes valid diversity. Meanwhile, early stopping methods lack domain semantics and failure attribution. Therefore, we propose DynSTEER to bridge this divide. DynSTEER performs stage-wise dynamic evaluation that adapts review capacity to weak or ambiguous stages. It compiles path-tolerant milestone graphs from available task inputs. Finally, it uses execution-time early stopping to eliminate wasteful computation.

3 Methodology

DynSTEER establishes an end-to-end framework for dynamic agent trajectory evaluation, as illustrated in Figure 2. Rather than treating rollouts as opaque black boxes or enforcing rigid single-path matching, DynSTEER coordinates three core components: stage-wise dynamic evaluation to resolve the evidence-granularity problem, which Sections 3.3 and 3.4 present; path-tolerant milestone-graph generation to preserve execution diversity, which Section 3.2 presents; and execution-time early termination to curb resource waste, which Section 3.4 presents.

3.1 Task Inputs and Dual-Clock Observation

Following interactive tool-using agent environments (Gu et al., 2024; Yang et al., 2023; Liu et al., 2024), a task provides instruction x , initial state $\mathcal{S}_0$, API catalog $\mathcal{A}$, and safety contracts $\mathcal{C}$. DynSTEER evaluates only the task inputs

$$V_{\text{task}} = (x, \mathcal{A}, \mathcal{I}_0), \quad (1)$$

where $\mathcal{I}_0$ is evaluator-visible initial-state metadata; hidden reference paths τ^* and terminal unit tests are never used for graph construction.

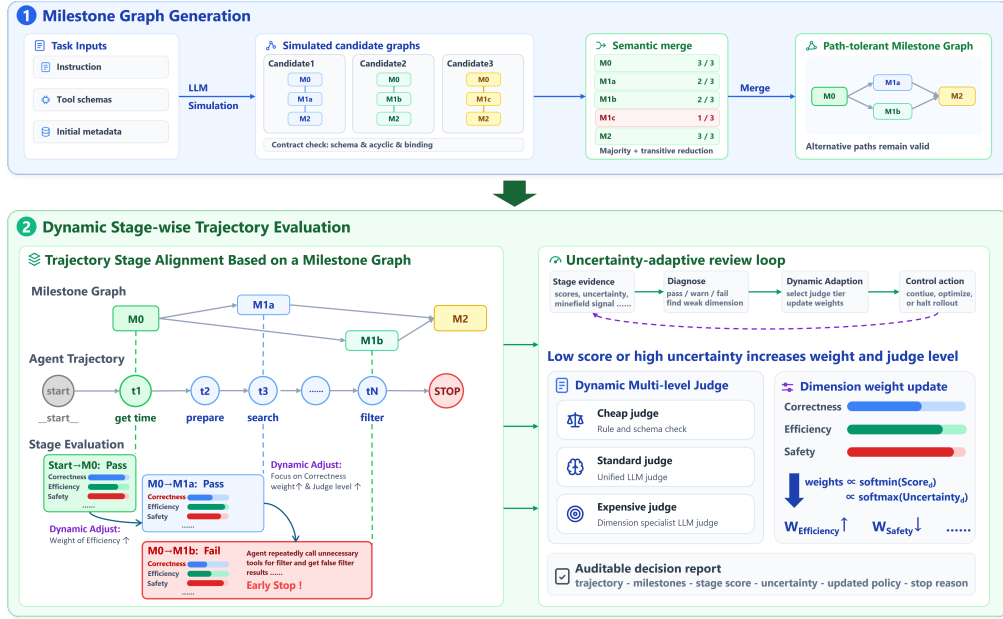


Figure 2: Overview of the DynSTEER Workflow. (1) Pre-execution milestone graph generation synthesizes multi-candidate simulated rollouts from task inputs to compile a path-tolerant DAG. (2) Execution-time stage-wise evaluation aligns closed actions, isolates dominator evidence windows to evaluate weak links with trajectory context, adaptively scales judge levels, and records executable early-stop decisions for subsequent evaluation control. During execution, the trace $\tau = \langle e_1, \dots, e_K \rangle$ is observed on two clocks. The raw event clock k tracks every event and triggers immediate fatal-action interception, whereas the closed-action clock t advances only after a tool return is observed and the agent yields control; milestone settlement and scoring use the closed-action clock t .

3.2 Milestone Graph Generation via Simulated Planning

To overcome single-reference bias, DynSTEER compiles V_{task} into a path-tolerant Milestone DAG $G = (M, E, \mathcal{K}, \mathcal{M}_{\text{fatal}})$ prior to execution, where M is the milestone set, E encodes precedence constraints, $\mathcal{K}$ specifies parameter bindings, and $\mathcal{M}_{\text{fatal}}$ defines hazardous operations.

Multi-Candidate Synthesis and Contract Validation. Given V_{task} , a blueprint generator synthesizes C candidate plans $\{G_1, \dots, G_C\}$ via temperature-sampled LLM prompting across diverse exploration heuristics, such as direct keyword queries versus hierarchical filtering. Each candidate $G_c = (M_c, E_c)$ undergoes deterministic contract verification:

$$\text{Valid}(G_c) = \text{SchemaCheck}(M_c, \mathcal{A}) \wedge \text{Acyclic}(E_c) \wedge \text{BindingConsistency}(\mathcal{K}_c) \quad (2)$$

Here, SchemaCheck requires every milestone and minefield to name an available API and to match its argument and return schema; Acyclic rejects circular precedence constraints; $\text{BindingConsistency}$ requires dynamic bindings to reference declared upstream outputs with compatible parameter types. A candidate failing any check is discarded. Appendix B.2 details these deterministic filters.

Strict-Majority Consensus and Transitive Reduction. To retain essential invariants while preserving exploratory flexibility, a milestone m is retained if and only if it appears in a strict majority of valid candidates:

$$M = \left\{ m \mid \sum_{c=1}^{C_{\text{valid}}} \mathbb{I}[m \in M_c] \geq \left\lceil \frac{C_{\text{valid}} + 1}{2} \right\rceil \right\} \quad (3)$$

Precedence edges are aggregated analogously. Finally, we compute the transitive reduction $E = \text{TR}(E_{\text{majority}})$ (Tarjan, 1972) to eliminate redundant shortcut dependencies, producing a minimal DAG that accommodates diverse legitimate execution strategies.

3.3 Stage-wise Dynamic Trajectory Evaluation

During execution, DynSTEER dynamically aligns closed actions against the milestone DAG. Algorithm 1 in Appendix B presents the complete algorithmic flow.

Ready Frontier Matching. Let $M_{<t}$ denote milestones settled prior to closed action t . The active search space is strictly bounded by the ready frontier:

$$\mathcal{F}_t = \{m \in M \setminus M_{<t} \mid \text{Pred}(m) \subseteq M_{<t}\} \quad (4)$$

An action a_t settles milestone $m^* \in \mathcal{F}_t$ if its tool call and state delta satisfy semantic predicates $\phi_{m^*}(a_t, s_t) = 1$. Restricting alignment to $\mathcal{F}_t$ prevents out-of-order settlement and reward gaming.

Dominator-Anchored Stage Isolation. To evaluate m^* without contamination from concurrent parallel branches, DynSTEER computes the immediate dominator $\text{idom}(m^*)$ in G (Lengauer & Tarjan, 1979), defined as the unique node that must be traversed on every directed path from start node s_0 to m^* . The stage evidence window is bounded by:

$$\mathcal{W}(m^*) = [t_{\text{idom}(m^*)}, t] \quad (5)$$

where $t_{\text{idom}(m^*)}$ is the execution step at which the dominator was settled. Isolating evidence to $\mathcal{W}(m^*)$ excludes unrelated actions from parallel branches while capturing all intermediate reasoning, trial-and-error attempts, and parameter adjustments leading directly to m^* . Unlike whole-trajectory evaluation, which obscures mistake locations, and single-step evaluation, which lacks context, dominator-anchored stages provide enough trajectory context to evaluate intermediate quality and pinpoint the exact first-error step.

Multidimensional Quality Scoring. Once settled, milestone m^* is scored across four orthogonal dimensions: correctness score c , efficiency score η , safety score s , and tool-compliance score κ . The composite stage score is:

$$S_t = \sum_{d \in \mathcal{D}} w_d^{(t)} s_d^{(t)}(m^*) \quad (6)$$

parameterized by dynamically normalized weights $\mathbf{w}^{(t)}$. Uncertainty-adaptive multi-level review then treats these stage scores and uncertainty estimates as control signals, allocating stronger review capacity to ambiguous or weak stages. This mechanism belongs to the stage-wise evaluator and completes its granularity solution, rather than being a separate early-stopping objective.

3.4 Uncertainty-Adaptive Review and Execution-Time Early Termination

Uncertainty-adaptive review is the control layer of stage-wise evaluation: it adjusts dimension weights and judge levels after each stage. Execution-time early termination is a separate downstream safeguard that reuses the same stage evidence to control resource waste.

Multi-Level Judge Structure. Rather than assigning a uniform evaluator across all stages, DynSTEER deploys a three-level judge hierarchy whose prompt templates appear in Appendix E:

- Level 1: Cheap Judge, denoted J_{cheap} : A deterministic structural evaluator inspecting tool status codes, schemas, and evidence rules without LLMs, incurring zero model inference overhead.
- Level 2: Standard Judge, denoted J_{std} : A multi-field unified LLM evaluator scoring all target dimensions simultaneously within a single structured JSON pass, aggregating confidence across multiple samples.
- Level 3: Expensive Judge, denoted J_{exp} : A single-dimension specialist LLM evaluator executing dedicated expert rubrics and deep counterfactual analysis across multiple deliberation passes to capture subtle behavioral defects.

Uncertainty-Adaptive Review Adjustment. DynSTEER dynamically shifts evaluation weights and escalates judge levels toward an agent’s vulnerable links. First, dimension

weights are updated via exponential gradient scaling:

$$w_d^{(t+1)} = \frac{w_d^{(t)} \cdot \exp\left(\alpha(1 - s_d^{(t)}) + \beta u_d^{(t)}\right)}{\sum_{d' \in \mathcal{D}} w_{d'}^{(t)} \cdot \exp\left(\alpha(1 - s_{d'}^{(t)}) + \beta u_{d'}^{(t)}\right)} \quad (7)$$

where $s_d^{(t)} \in [0, 1]$ is the dimension score, $u_d^{(t)} \in [0, 1]$ is dimension uncertainty, and α, β scale sensitivity to quality deficits and uncertainty. Appendix C gives the design rationale and mathematical derivation of this update. Second, the baseline judge level is set to:

$$\text{Level}_{\text{base}}^{(t+1)} = \begin{cases} J_{\text{cheap}}, & \text{if } S_t \geq \theta_{\text{pass}} + \delta_{\text{margin}} \wedge \max_d u_d^{(t)} \leq \theta_{\text{low_u}} \wedge \mathcal{M}_{\text{fatal}}^{(t)} = 0 \\ J_{\text{std}}, & \text{otherwise} \end{cases} \quad (8)$$

For each individual dimension d , its level $\text{Level}_d^{(t+1)}$ adaptively escalates along $J_{\text{cheap}} < J_{\text{std}} < J_{\text{exp}}$:

$$\text{Level}_d^{(t+1)} = \begin{cases} J_{\text{exp}}, & \text{if } s_d^{(t)} < \theta_{\text{fail}} \\ \min(J_{\text{exp}}, \text{Level}_d^{(t)} + 1), & \text{if } s_d^{(t)} < \theta_{\text{warn}} \vee u_d^{(t)} \geq \theta_{\text{high_u}} \\ \text{Level}_{\text{base}}^{(t+1)}, & \text{otherwise} \end{cases} \quad (9)$$

This focuses expensive LLM deliberation selectively on weak or ambiguous execution links while relaxing confident stages to rule-based verification in J_{cheap} to minimize unnecessary LLM invocations. Together with stage segmentation and local evidence isolation, this review strategy resolves the evidence-granularity problem. The next mechanism addresses a distinct objective: preventing unrecoverable prefixes from consuming further execution resources.

Execution-Time Early Termination. To prevent runaway executions on unrecoverable trajectories, DynSTEER monitors four stopping criteria. First, Fatal Minefield Interception, where $e_k \in \mathcal{M}_{\text{fatal}}$, signals immediately on the raw event clock upon hazardous or irreversible actions. Second, Structural Invariant Failures catch persistent unparseable payloads or repeated schema errors. Third, Persistent Low Quality, where $S_t < \theta_{\text{fail}}$, signals when accumulated stage deficits indicate unrecoverable goal drift. Fourth, Frontier Stagnation signals when an agent takes N_{stag} consecutive actions without advancing the ready frontier $\mathcal{F}_t$, detecting pathological loops. A stop emits an auditable report recording the step, dominator anchor, and causal failure dimension. In our controlled experiments, we replay the decision against a complete source trajectory: the report defines a virtual stop boundary, and we measure efficiency by matching that boundary to the recorded prefix of the source execution.

4 Experiments

We evaluate DynSTEER on ToolSandbox and SWE-bench Pro using the same set of instruction-tuned agent backbones. The experiments address four central research questions. RQ1 asks whether stage-wise dynamic evaluation provides superior error focus and model discriminability over native terminal evaluation. RQ2 asks how much dynamic judge levels, dynamic dimension weighting, and execution-time early stopping contribute to discriminability and efficiency. RQ3 asks whether milestone graphs compiled from task inputs reliably capture necessary goals while accommodating diverse solution paths. RQ4 asks how effectively execution-time early stopping identifies and removes wasteful execution prefixes.

4.1 Experimental Setup

Benchmark Environments and Evaluated Models. We evaluate on two complementary agent settings. ToolSandbox (Gu et al., 2024) provides stateful tool environments, dynamic argument types, and explicit safety minefield invariants. SWE-bench Pro (Deng et al., 2025) contains long-horizon software-engineering tasks that require repository understanding, multi-file edits, and test-based outcome verification. Appendix A reports the

Table 1: Main Evaluation Results on ToolSandbox and SWE-bench Pro. Default and DynSTEER use identical trajectories; Saved Steps is computed per model.

Benchmark	Evaluator	Model	Mean Score $\uparrow$	Stop Rate	DS ($\epsilon=1.00$) $\uparrow$	Saved Steps $\uparrow$
SWE-bench Pro	Default	DeepSeek-V4-Flash	69.54 $\pm$ 8.60	0.00%	0.0324	0.00%
		DeepSeek-V4-Pro	63.64 $\pm$ 8.93	0.00%		0.00%
		Qwen3-Max-2026-01	66.10 $\pm$ 4.15	0.00%		0.00%
		Qwen-Plus-2025-12	64.03 $\pm$ 3.96	0.00%		0.00%
	DynSTEER	DeepSeek-V4-Flash	77.40 $\pm$ 5.50	6.12%	0.0857	–
		DeepSeek-V4-Pro	69.90 $\pm$ 0.62	9.00%		–
		Qwen3-Max-2026-01	64.93 $\pm$ 3.98	11.00%		–
		Qwen-Plus-2025-12	61.87 $\pm$ 2.87	7.07%		–
ToolSandbox	Default	DeepSeek-V4-Flash	78.80 $\pm$ 0.43	0.00%	0.0270	0.00%
		DeepSeek-V4-Pro	77.18 $\pm$ 0.14	0.00%		0.00%
		Qwen3-Max-2026-01	73.68 $\pm$ 1.00	0.00%		0.00%
		Qwen-Plus-2025-12	73.62 $\pm$ 1.09	0.00%		0.00%
	DynSTEER	DeepSeek-V4-Flash	73.38 $\pm$ 0.26	16.63%	0.0500	11.44%
		DeepSeek-V4-Pro	71.11 $\pm$ 0.21	17.68%		16.06%
		Qwen3-Max-2026-01	65.03 $\pm$ 0.94	27.24%		21.54%
		Qwen-Plus-2025-12	66.15 $\pm$ 0.96	18.53%		22.14%

dataset statistics and SWE-bench Pro case-selection protocol. We evaluate DeepSeek-V4-Pro, DeepSeek-V4-Flash (DeepSeek-AI et al., 2024)¹, Qwen3-Max-2026-01-23, and Qwen-Plus-2025-12-01 (Yang et al., 2024) under standard ReAct prompting (Yao et al., 2023).

Comparative Paradigms. For controlled comparison, Default and DynSTEER are evaluated on the same source trajectory for every model, repeat, and scenario. Default applies the native terminal verifier, whereas DynSTEER applies stage-wise evaluation, dominator review, and adaptive judging to the recorded trajectory. When early termination fires, DynSTEER records a virtual stop boundary rather than executing a live interruption; all early-stop savings therefore represent paired counterfactual savings.

Evaluation Metrics. We assess performance across three dimensions:

- **Metric Discriminability:** Following evaluation meta-evaluation literature (Sakai, 2006; 2014; Carterette, 2012; Zheng et al., 2023), we measure Discriminability Score, denoted DS, at margin ϵ :

$$DS(\epsilon) = \left(\frac{\sigma_{\text{pop}}}{\mu} \right) \cdot \sqrt{\frac{N_{\text{sig}}(\epsilon)}{\binom{M}{2}}}, \quad N_{\text{sig}}(\epsilon) = \sum_{1 \leq i < j \leq M} \mathbb{I}(|\bar{s}_{m_i} - \bar{s}_{m_j}| > \epsilon) \quad (10)$$

where μ is the population mean score across models, σ_{pop} is the standard deviation across model means, and $N_{\text{sig}}(\epsilon)$ is the number of statistically distinguishable model pairs at margin ϵ . The ratio σ_{pop}/μ represents the Coefficient of Variation that penalizes metric compression. For readability, we normalize agent scores to $[0, 100]$ and report all DS results with $\epsilon = 1.00$.

- **Milestone Graph Quality:** On ToolSandbox, we compute Tool Operation Micro-F1 and Macro-F1 to measure semantic tool coverage, Fatal Minefield F1 for safety-critical action detection, and Graph Edit Distance similarity, or GED, for structural topology alignment.
- **Efficiency and Early Termination:** For each model, Saved Steps is the percentage of matched Default full-trajectory steps avoided by DynSTEER policy stops; non-stopped runs contribute zero savings. Stop Rate divides policy stops by the matched evaluation population and includes persistent low quality, frontier stagnation, and fatal minefields.

4.2 RQ1: Stage-wise Dynamic Evaluation, Error Focus, and Model Discriminability

Discrimination and efficiency. DynSTEER increases DS from 0.0270 to 0.0500 on ToolSandbox and from 0.0324 to 0.0857 on SWE-bench Pro. All six model pairs become separated, and the standard deviation of model means rises from 2.24 to 3.45 and from 2.34 to 5.87, respectively. Policy stopping also saves 11.44–22.14% of Default steps across ToolSandbox models (17.74% overall) and 8.40% on SWE-bench Pro. These savings are conservative because non-stopped runs count as zero; DynSTEER removes prefixes judged unrecoverable without collapsing model separation.

¹The experiments use the post-2026-07-31 version of DeepSeek-V4-Flash.

Table 2: Component Ablation on a ToolSandbox Subset. The ablation uses a ToolSandbox subset stratified by task type and safety category. DS uses $\epsilon=1.00$. Default uses the native terminal score, whereas the other rows use DynSTEER overall scores. Saved Steps is the aggregate avoided steps divided by the total Default full-trajectory steps, shown as a percentage.

Configuration	DS ($\epsilon=1.00$) $\uparrow$	Stop Rate	Saved Steps
Default	0.0228	0.00%	—
DynSTEER	0.0528	18.92%	12.90%
-w/o dynamic weighting	0.0414	17.92%	12.75%
-w/o dynamic judge level	0.0439	15.83%	12.58%
-w/o dynamic	0.0311	15.83%	12.49%
-w/o policy stop	0.0286	0.00%	—

Stability and rank consistency. The mean repeat-level standard deviation decreases from 0.66 to 0.59 on ToolSandbox and from 6.41 to 3.24 on SWE-bench Pro. Within each benchmark, DynSTEER preserves five of six pairwise model orders on ToolSandbox (Kendall $\tau = 0.67$) and four of six on SWE-bench Pro ($\tau = 0.33$); DeepSeek-V4-Flash remains first in both cases. Thus, larger between-model spread coincides with lower repeat noise and broadly stable rankings.

Process sensitivity. ToolSandbox means decrease as DynSTEER penalizes retries, parameter hallucinations, and hazardous side effects that still reach a correct terminal state, whereas SWE-bench Pro adjustments are model-dependent. In blind auditing of trajectories with divergent labels, DynSTEER localized the first critical error with 91.7% expert agreement; terminal verification found none.

4.3 RQ2: Component Contributions to Discriminability and Early Stopping

To answer RQ2, we conduct a component ablation on a ToolSandbox subset stratified by task type and safety category; Appendix A details the sampling protocol. All configurations use identity-matched trajectories. From Table 2, removing any dynamic component or the early-stopping mechanism hurts DS, Stop Rate, and Saved Steps relative to full DynSTEER.

Early stopping has the largest single-component impact. Disabling policy stopping reduces DS from 0.0528 to 0.0286 and removes both stopping and step savings. A plausible reason is that early stopping converts an unrecoverable execution prefix into decisive failure evidence. Without that signal, a long routine suffix can dilute the score distribution and make systematically weak executions resemble recoverable noise.

Dynamic weighting and judge levels contribute complementary separation signals. Dynamically adjusting dimension weights within stage evaluation makes models easier to distinguish: it reallocates evidence toward low-scoring, high-uncertainty dimensions and prevents uniform averaging from canceling complementary weaknesses. Dynamic judge levels nevertheless provide strong discriminability: relative to stage-only evaluation (-w/o dynamic), retaining judge-level escalation under -w/o dynamic weighting raises DS from 0.0311 to 0.0414. The early-stop statistics further separate the two mechanisms: -w/o dynamic weighting attains a higher Stop Rate and Saved Steps than -w/o dynamic judge level. This suggests that stronger review of poorly executed dimensions is more likely to expose problems and assign low scores promptly, allowing the agent to stop before additional wasted steps accumulate.

4.4 RQ3: Blueprint Compilation Reliability and Path Tolerance

Table 3 summarizes the semantic accuracy, structural distance, and safety alignment of generated milestone DAGs against canonical expert annotations. DynSTEER’s multi-candidate synthesis and strict-majority consensus achieve an 86.0% Tool Operation Micro-F1 with 81.7% precision and 90.7% recall, and an 86.7% Fatal Minefield F1 with 86.0% recall, reliably identifying irreversible operations from available task inputs alone. Transitive reduction retains alternative parallel branches without penalizing valid exploratory variations, such as searching by contact name rather than date, achieving a 100% valid DAG compilation rate with consistent topological alignment.

Table 3: Milestone Blueprint Compilation Reliability. Evaluation of automatically compiled milestone DAGs against expert reference graphs on ToolSandbox.

Evaluation Dimension	Precision	Recall	F1 Score
Tool Operation (Micro)	81.7%	90.7%	86.0%
Tool Operation (Macro)	72.7%	66.0%	66.7%
Fatal Minefield Detection	87.5%	86.0%	86.7%
Structural Topology Metric		Score	
Topological GED Similarity		0.493	
Valid DAG Compilation Rate		100.0%	

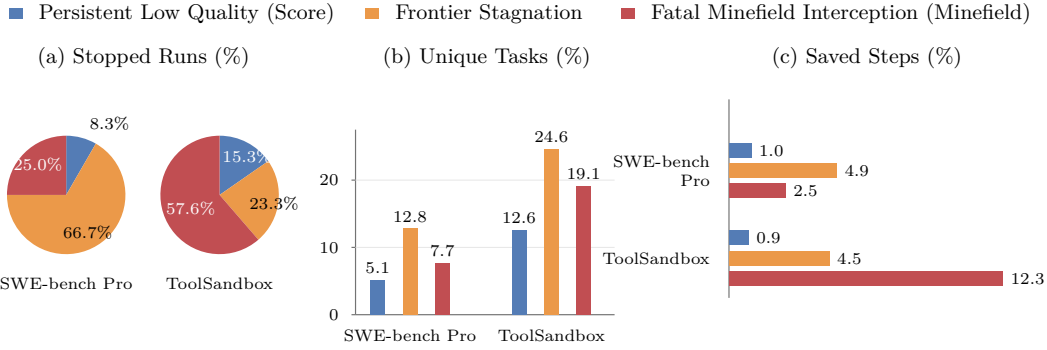


Figure 3: Early-stop behavior and savings by termination criterion. (a) Stopped-run shares, (b) unique-task coverage, and (c) Saved Steps contributions on ToolSandbox and SWE-bench Pro. Unique-task shares are not additive.

4.5 RQ4: Early-Stop Behavior and Counterfactual Savings

Benchmark-dependent failure modes. Figure 3(a) shows that Fatal Minefield Interception dominates ToolSandbox stopped runs at 57.56%, and Figure 3(c) shows its 12.34% contribution to the aggregate 17.74% Saved Steps. Its unique-task share is only 19.06%, whereas Frontier Stagnation covers the largest unique-task share (24.56%) but saves only 4.53%. Thus, ToolSandbox failures are often early unsafe or invalid calls on interactive tasks.

On SWE-bench Pro, Frontier Stagnation dominates stopped runs at 66.67% and unique-task coverage at 12.82%, whereas Fatal Minefield Interception accounts for 25.00% and 7.69%, respectively (Figure 3(a,b)). Frontier Stagnation contributes the largest saving, 4.86% of the aggregate 8.40%, reflecting prolonged repository exploration and repair loops before a terminal test result. Persistent Low Quality remains conservative, saving 1.00% on SWE-bench Pro and 0.87% on ToolSandbox. In addition, Appendix D presents cases illustrating auditable stop decisions.

5 Conclusion

In this work, we propose DynSTEER, a dynamic stage-wise framework that advances LLM agent trajectory evaluation from passive, post-hoc terminal verification toward proactive execution review. DynSTEER segments rollouts at key execution nodes, adapts its multi-level review strategy to stage-level results, compiles a path-tolerant milestone graph without reference leakage, and terminates unrecoverable executions. Across ToolSandbox and SWE-bench Pro, DynSTEER improves discriminability by 85.2% and 164.0% and saves 17.74% and 8.40% of execution steps, providing a principled and cost-aware foundation for LLM-agent evaluation and optimization.

References

- Javed A Aslam, Virgiliu Pavlu, and Emine Yilmaz. A statistical method for system evaluation using incomplete judgments. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 541–548, 2006.
- Ben Carterette. Multiple testing in statistical analysis of systems-based information retrieval experiments. *ACM Transactions on Information Systems (TOIS)*, 30(1):1–34, 2012.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Chen, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- Jian Gu, Shusen Jiang, Boyi Zhang, Zeqing Meng, Yifan Lu, Jiarui Gao, Yuchen Hu, Shengyu Zhang, and Fei Wu. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. *arXiv preprint arXiv:2408.04682*, 2024.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Kevin Kuo, Ethan Anderson, and Dawn Song. Do agents dream of root shells? partial-credit evaluation of LLM agents in capture the flag challenges. In *IEEE/ACM International Conference on AI Engineering (AIWare)*, 2026.
- Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 1(1):121–141, 1979.
- Meng Li, Yifan Sun, Dong Zhao, Yue Kang, and Xin Wang. TRAJECT-Bench: A trajectory-aware benchmark for evaluating agentic tool use. *arXiv preprint arXiv:2510.04550*, 2025.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hanghang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.
- Shishir G Patil, Huanru Zhang, Zheyuan Yan, and Joseph E Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning (ICML)*, 2025.
- Chen Qian, Yuxuan Dang, Jiaao Chen, Wei Liu, Yue Wang, Min Zhang, and Heng Ji. AgentProcessBench: Evaluating step-level effectiveness in tool-augmented agent trajectories. *arXiv preprint arXiv:2410.05284*, 2024.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024.
- Tetsuya Sakai. Evaluating evaluation metrics based on discriminative power. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 525–532, 2006.
- Tetsuya Sakai. Statistical significance, power, and discriminability of evaluation metrics. *Information Retrieval*, 17(5-6):430–457, 2014.
- Yuling Shi, Zhensu Sun, Junsen Dong, Chengcheng Wan, David Lo, and Xiaodong Gu. EarlyEval: Cheaper agent evaluation via early outcome prediction. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026.

- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 8634–8652, 2023.
- Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.
- Dingmin Wu, Ce Zhang, Chen Lin, Min Chen, and Rui Song. Agent-testing agent: A meta-agent for automated testing and evaluation of conversational AI agents. In *Proceedings of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026.
- An Yang, Baosong Yang, Binyuan Zhang, Bofei Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- John Yang, Syeda Nahida Akter, Chou Han, Noah Shinn, Ofir Press, Karthik Narasimhan, and Shunyu Yao. InterCode: Standardizing and benchmarking interactive coding with execution feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 36: 15886–15915, 2023.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Yuxuan Zhang, Jiaqi Wang, Zihan Liu, Hao Chen, Bo Li, and Jing He. Claw-Eval: Towards trustworthy evaluation of LLM agents. *arXiv preprint arXiv:2604.06132*, 2026.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 46595–46623, 2023.
- Daniel Zhou, Vikram Sreedhar, and Graham Neubig. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories. *arXiv preprint arXiv:2504.08942*, 2025.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Table 4: ToolSandbox Benchmark Dataset Statistics. Distribution of evaluated scenarios across functional domains, interaction modalities, adversarial perturbation variants, and safety minefield configurations.

Scenario Domain / Sub-category	Count	Avg. User Turns	Available Tools	Minefield Invariants
Calendar & Scheduling Management	112	3.4	8 – 18	Data integrity, date bounds
Messaging & Notification Services	136	2.9	6 – 15	Privacy, unverified sends
Contact & Personal CRM	98	2.6	5 – 12	Irreversible deletion
Financial & Transaction Processing	84	4.1	10 – 24	Unauthorized transfer
Information Retrieval & Weather/Geocoding	79	3.2	7 – 16	Parameter type constraints
Perturbation & Difficulty Variants	Count	Properties & Challenge Characteristics		
Standard Canonical	127	Nominal parameters, standard system prompt and API schemas		
Distraction Tools (3–10 extra APIs)	158	Noise tools added to prompt to test agent tool retrieval		
Scrambled Arguments / Descriptions	142	Shuffled arg orders and adversarial field descriptors		
Insufficient Information (Rejection Tasks)	82	Ambiguous queries requiring clarification or safe refusal		
Total Evaluated Benchmark Pool	509	3.2 turns	12.4 tools/task	704 total minefields

A Benchmark Dataset Details

This appendix documents the two benchmark populations and the protocol used to obtain the SWE-bench Pro evaluation subset.

ToolSandbox. ToolSandbox (Gu et al., 2024) is an interactive, stateful benchmark specifically engineered to evaluate complex multi-turn decision-making capabilities of tool-using LLM agents. Unlike static benchmarks that rely solely on mock API dictionaries, ToolSandbox maintains a stateful world environment with persistent entities, temporal relationships, and strict data invariants.

Methodological Selection of Sandboxed Scenarios. In our evaluation, we benchmark agents across the suite of 509 scenarios that operate within fully sandboxed, local execution backends rather than relying on live commercial RapidAPI endpoints. This methodological choice is grounded in three experimental considerations:

1. **Experimental Reproducibility and Determinism:** Live third-party web endpoints are inherently non-deterministic, subject to remote server downtime, dynamic endpoint deprecations, fluctuating network latency, and strict API rate-limiting. Self-contained, sandboxed tool-execution backends ensure deterministic experimental reproducibility across independent evaluation trials.
2. **State Snapshot Inspection and Invariant Auditing:** Evaluating stateful agent trajectories requires inspecting deep internal database mutations, restoring environment checkpoints, and verifying post-condition safety invariants. Purely local execution backends permit exact, serializable state snapshot verification, which is impossible with black-box commercial cloud endpoints.
3. **Hermetic Execution Security and Zero Contamination:** Running large-scale evaluations on live commercial APIs introduces uncontrolled recurring monetary costs and risks data leakage over external public networks. The sandboxed suite provides a secure, self-contained testbed with zero external dependencies.

Table 4 provides descriptive statistics for the 509 sandboxed scenarios:

- **Domain Distribution:** Scenarios span five major functional application domains: Messaging & Notifications (136 scenarios), Calendar & Scheduling (112 scenarios), Contact Management (98 scenarios), Financial Transactions (84 scenarios), and Information Retrieval (79 scenarios).
- **Multi-Turn Dialogues:** Each scenario involves an average of 3.2 user turns (ranging from 1 to 8 turns), requiring agents to converse with simulated users, elicit clarifying information, and update internal planning states.
- **Tool Environment Complexity:** Scenarios provide an average of 12.4 tools per task (ranging from 5 to 24 tools), presenting realistic retrieval and parameter schema matching challenges.

Table 5: SWE-bench Pro Public-Split Sampling Statistics. The evaluated subset is stratified by repository to preserve the public test split’s project diversity.

Repository	Public Split	Stratified Sample
NodeBB/NodeBB	44	6
ansible/ansible	96	14
element-hq/element-web	56	7
flipt-io/flipt	85	12
future-architect/vuls	62	8
gravitational/teleport	76	11
internetarchive/openlibrary	91	13
navidrome/navidrome	57	7
protonmail/webclients	65	9
qutebrowser/qutebrowser	79	11
tutao/tutanota	20	2
Total	731	100

- **Perturbation Variants:** The benchmark incorporates four difficulty variants: canonical tasks (127 scenarios), distraction tool injection (158 scenarios), scrambled parameter descriptors (142 scenarios), and insufficient information queries (82 scenarios) where agents must safely refuse or ask clarifying questions rather than invoking tools blindly.
- **Safety Minefields:** The dataset contains 704 explicit minefield invariants across the 509 scenarios, encompassing irreversible database deletions, privacy leaks, and invalid transaction commitments.

SWE-bench Pro. SWE-bench Pro (Deng et al., 2025) supplies repository-level software-engineering tasks with Docker execution environments, gold patches, and test-based terminal verification. We use its public test split, whose local snapshot contains 731 tasks from 11 repositories. To bound execution time, compute cost, and image-transfer cost, we first draw a repository-stratified subset; Table 5 shows the resulting repository distribution.

During execution, Docker images for some selected tasks could not be pulled from the available registry. These infrastructure failures do not measure agent capability, so we remove the affected Default and DynSTEER pairs before computing scores, stop rates, DS, and early-stop statistics. After this exclusion, 39 matched tasks remain in the SWE-bench Pro analysis. This retained subset is a material limitation: it preserves repository stratification at sampling time but does not provide the coverage of the complete public split.

Ablation Subset. To control evaluation time and compute cost, the component ablation samples 100 ToolSandbox scenarios from the 509 sandboxed scenarios using stratification by task type and safety category. Unless otherwise stated, this subset follows the same evaluated models, repeat protocol, and paired-comparison design as the main experiment.

B Algorithmic Formulations and Proof Details

B.1 DynSTEER Execution-Time Stage Evaluation and Control Algorithm

Algorithm 1 formalizes the complete execution-time stage evaluation, dominator evidence window extraction, uncertainty-adaptive judge-level adaptation, and early termination protocol, matching our implementation in `dynsteer/evaluate/evaluator.py` and `policy.py`.

B.2 Candidate Contract Validation

Before consensus, every candidate is checked deterministically; no LLM is consulted during this filtering step. Schema validation rejects malformed graphs and unknown operations: each milestone must have a unique identifier, a well-formed success condition, and a canonical tool name present in $\mathcal{A}$; minefield descriptors must likewise reference available destructive

Algorithm 1 DynSTEER Execution-Time Stage Evaluation and Control

Require: Task inputs V_{task} , candidate count C , stop thresholds Θ
 Ensure: Final evaluation report $\mathcal{R}$, termination status

- 1: $\{G_1, \dots, G_C\} \leftarrow \text{SynthesizeCandidates}(V_{\text{task}}, C)$
- 2: $G \leftarrow \text{ValidateAndAggregate}(\{G_1, \dots, G_C\})$ {Milestone DAG $G = (M, E, \mathcal{K}, \mathcal{M}_{\text{fatal}})$ }
- 3: $M_{\text{settled}} \leftarrow \emptyset$, $t \leftarrow 0$, $\mathcal{R} \leftarrow \emptyset$, $\mathbf{w}_1 \leftarrow \text{InitWeights}()$, $\text{Level}^{(1)} \leftarrow J_{\text{cheap}}$
- 4: for each event e_k in agent execution rollout do
- 5: if $e_k \in \mathcal{M}_{\text{fatal}}$ then
- 6: return $\text{Halt}(\mathcal{R}, \text{reason} = \text{"Fatal Minefield"}, k)$
- 7: end if
- 8: if e_k is Closed Agent Action a_t then
- 9: $t \leftarrow t + 1$
- 10: $\mathcal{F}_t \leftarrow \{m \in M \setminus M_{\text{settled}} \mid \text{Pred}(m) \subseteq M_{\text{settled}}\}$
- 11: $m^* \leftarrow \text{MatchMilestone}(\mathcal{F}_t, a_t, s_t)$
- 12: if $m^* \neq \text{None}$ then
- 13: $M_{\text{settled}} \leftarrow M_{\text{settled}} \cup \{m^*\}$
- 14: $\mathcal{W} \leftarrow [t_{\text{idom}(m^*)}, t]$ {Dominator-anchored evidence window}
- 15: $S_t, \mathbf{s}_t, \mathbf{u}_t, \mathbf{diag} \leftarrow \text{EvaluateStage}(\mathcal{W}, m^*, \mathbf{w}_t, \text{Level}^{(t)})$
- 16: $\mathcal{R} \leftarrow \mathcal{R} \cup \{(m^*, S_t, \mathbf{diag})\}$
- 17: $\mathbf{w}_{t+1}, \text{Level}^{(t+1)} \leftarrow \text{UpdateEvaluationPolicy}(\mathbf{w}_t, \mathbf{s}_t, \mathbf{u}_t, S_t, \Theta)$ {Dynamic strategy adaptation}
- 18: end if
- 19: if $\text{CheckEarlyStopCriteria}(\mathcal{R}, \mathcal{F}_t, S_t, t, \Theta)$ then
- 20: return $\text{Halt}(\mathcal{R}, \text{reason} = \text{"Early Termination"}, t)$
- 21: end if
- 22: end if
- 23: end for
- 24: return $\text{FinalizeReport}(\mathcal{R}, M_{\text{settled}})$

operations. It also checks that edge endpoints, dependency declarations, and output-field references exist.

Structural validation verifies that the directed precedence relation is finite, free of self-loops and cycles, and consistent with milestone reachability. Edges that duplicate an existing dependency are collapsed before aggregation, while genuinely distinct precedence constraints are retained.

Binding validation requires every dynamic parameter binding to name a milestone that can execute earlier in the same candidate, an output field declared by that milestone, and a target parameter whose schema accepts the referenced type. Literal arguments are checked against required keys and declared types. Thus, hallucinated APIs, malformed payloads, cyclic plans, and dangling or type-incompatible bindings are removed before majority consensus.

B.3 Blueprint Consensus Aggregation Details

The blueprint aggregation pipeline synthesizes multiple candidate graphs $\{G_1, \dots, G_C\}$ into a canonical Milestone DAG $G = (M, E, \mathcal{K}, \mathcal{M}_{\text{fatal}})$.

Let each candidate graph be represented by its set of nodes M_c , directed precedence edges E_c , and tool argument bindings $\mathcal{K}_c$. We define node semantic equivalence $\sim$ using canonical tool names and parameter role bindings:

$$m_i \sim m_j \iff \text{ToolName}(m_i) = \text{ToolName}(m_j) \wedge \text{Role}(m_i) = \text{Role}(m_j) \quad (11)$$

For each equivalence class $[m]$, its consensus support is:

$$\text{Support}([m]) = \sum_{c=1}^{C_{\text{valid}}} \mathbb{I}(\exists m' \in M_c \text{ s.t. } m' \in [m]) \quad (12)$$

Milestones with $\text{Support}([m]) \geq \lceil (C_{\text{valid}} + 1)/2 \rceil$ are admitted into the canonical milestone set M . Precedence edges are aggregated analogously, and transitive reduction $\text{TR}(E)$ is com-

puted using Tarjan’s linear-time topological ordering algorithm (Tarjan, 1972) to eliminate redundant shortcut edges.

B.4 Immediate Dominator Stage Anchoring Proof

Let $G = (M, E)$ be the DAG with augmented entry node s_0 . A node d dominates node n ($d \text{ dom } n$) if every directed path from s_0 to n contains d . The immediate dominator $\text{idom}(n)$ is the unique node that strictly dominates n without dominating any other strict dominator of n . We compute the dominator tree using the Lengauer-Tarjan algorithm (Lengauer & Tarjan, 1979) in $\mathcal{O}(|E|\alpha(|E|, |M|))$ time.

When milestone m^* is settled at step t , its stage evidence window $\mathcal{W}(m^*) = [t_{\text{idom}(m^*)}, t]$ encompasses precisely the intermediate steps executed between its immediate dominator and m^* . Because m^* could not have been reached without traversing $\text{idom}(m^*)$, all causal dependencies are preserved while concurrent, independent branches are cleanly partitioned.

C Theoretical Design of the Dynamic Weighting Mechanism

The evaluator must repeatedly direct attention toward dimensions whose recent evidence reveals either low quality or high uncertainty, while preserving positivity and comparability across stages. A normalized exponential tilt is a natural mechanism for this purpose: it turns each observed signal into a multiplicative potential rather than an ad hoc reassignment, so persistent evidence is accumulated rather than overwritten. Starting from Eq. equation 7, the common factor $\exp(\alpha)$ cancels after normalization and gives

$$w_d^{(t+1)} \propto w_d^{(t)} \exp\left(-\alpha s_d^{(t)} + \beta u_d^{(t)}\right). \quad (13)$$

Thus each stage multiplies a dimension’s weight by the exponential of the dimension-specific potential $-\alpha s_d^{(t)} + \beta u_d^{(t)}$. Lower scores increase the potential’s contribution to future attention, and higher uncertainty does the same.

Iterating this multiplicative rule makes the cumulative nature of the mechanism explicit. Because normalization introduces only a dimension-independent constant, the ratio between any two dimensions after T stages is

$$\frac{w_d^{(T+1)}}{w_{d'}^{(T+1)}} = \frac{w_d^{(1)}}{w_{d'}^{(1)}} \exp\left[\alpha \sum_{t=1}^T (s_{d'}^{(t)} - s_d^{(t)}) + \beta \sum_{t=1}^T (u_d^{(t)} - u_{d'}^{(t)})\right]. \quad (14)$$

Consequently, a linearly growing cumulative score deficit produces an exponentially growing weight ratio. This is the soft-min form of attention: the mechanism does not merely remember the latest weak dimension, but progressively reallocates review capacity toward persistent weaknesses.

To understand what such repeated tilting does to a model’s aggregate score, first isolate the score component by freezing a profile $s = (s_1, \dots, s_D)$. Define the partition function, tilted distribution, and tilted score by

$$Z(\lambda) = \sum_{d \in \mathcal{D}} e^{-\lambda s_d}, \quad p_d(\lambda) = \frac{e^{-\lambda s_d}}{Z(\lambda)}, \quad S(\lambda) = \sum_{d \in \mathcal{D}} p_d(\lambda) s_d. \quad (15)$$

The parameter λ represents the effective score sensitivity accumulated over repeated updates. Differentiating log Z gives

$$\frac{\partial}{\partial \lambda} \log Z(\lambda) = \frac{\sum_d (-s_d) e^{-\lambda s_d}}{Z(\lambda)} = -S(\lambda), \quad (16)$$

and differentiating once more yields

$$\frac{dS}{d\lambda} = -\frac{d^2}{d\lambda^2} \log Z(\lambda) = -\text{Var}_{p(\lambda)}(s) \leq 0. \quad (17)$$

This equation contains both the direction and the rate of the score change. Every model's tilted score decreases as focus sharpens, but the decrease is proportional to the weighted variance of its own dimension profile. A flat profile changes little, whereas a profile with a distinct weakness changes sharply.

The next question is what information this tilt exposes. Decompose model i 's profile into an average level and a zero-sum deviation:

$$s_i = m_i \mathbf{1} + v_i, \quad \sum_{d \in \mathcal{D}} v_{i,d} = 0. \quad (18)$$

Uniform weighting retains only the first component:

$$S_i^{\text{avg}} = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} (m_i + v_{i,d}) = m_i. \quad (19)$$

It therefore cancels v_i , even though v_i records where a model succeeds and fails. In the sharp-tilt limit, only the weakest dimension receives weight, so

$$\lim_{\lambda \rightarrow \infty} S_i(\lambda) = \min_{d \in \mathcal{D}} (m_i + v_{i,d}) = m_i + \min_{d \in \mathcal{D}} v_{i,d}. \quad (20)$$

The second term is the depth of the model's worst weakness relative to its own average. For two models A and B , the limiting gap becomes

$$S_A(\infty) - S_B(\infty) = \underbrace{(m_A - m_B)}_{\text{average-level difference}} + \underbrace{\left(\min_d v_{A,d} - \min_d v_{B,d} \right)}_{\text{weakness-depth difference}}. \quad (21)$$

Hence the mechanism changes the comparison axis: uniform weighting compares average levels, whereas exponential tilting progressively adds the information carried by complementary failure patterns. For example, $s_A = (0.9, 0.3)$ and $s_B = (0.5, 0.9)$ give a uniform gap of 0.100, but the limiting gap becomes 0.200 even though both tilted scores decrease. Both models are pulled downward, while the model with the deeper weakness is pulled down faster.

The same reasoning also clarifies when separation is guaranteed. Let $\mathcal{I}$ denote the model set and define the cross-model dispersion

$$\text{Dis}(\lambda) = \sum_{i \in \mathcal{I}} (S_i(\lambda) - \bar{S}(\lambda))^2, \quad \bar{S}(\lambda) = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} S_i(\lambda), \quad (22)$$

where p_i is the tilt in Eq. equation 15 applied to s_i . Using Eq. equation 17,

$$\frac{d \text{Dis}}{d\lambda} = 2 \sum_{i \in \mathcal{I}} (S_i - \bar{S}) \frac{dS_i}{d\lambda} = -2 \sum_{i \in \mathcal{I}} (S_i - \bar{S}) \text{Var}_{p_i}(s_i). \quad (23)$$

Dispersion increases when models below the current tilted mean have larger internal variances, that is, when their disadvantages are concentrated in deeper weak links. If instead a high-average model also contains the deepest weakness, this derivative can become negative and the gap can compress. The mechanism therefore guarantees a principled shift of evidence toward weak links, not unconditional divergence between all model pairs.

The uncertainty term now completes the design. In Eq. equation 14, $\beta u_d^{(t)}$ contributes another dimension-specific potential: uncertain evidence receives attention before it is discarded, allowing the next review level to test an apparent weakness with stronger evidence. Once that evidence is resolved, subsequent updates respond to the refined score and uncertainty estimates. Taken together, the score tilt supplies the comparative objective—moving from average performance to weakness depth—while uncertainty controls the epistemic allocation of review capacity during that transition.

D Qualitative Case Studies and Error Analysis

D.1 Case 1: Early Termination of Infinite Search Frontier Stagnation

Scenario ID:

search_message_with_recency_oldest_multiple_user_turn_alt_3_distraction_tools_arg_type_scrambled

Agent Model: Qwen3-Max-2026-01 under ReAct prompting.

Execution Trace Summary:

- The user asks for the oldest matching text. The agent asks one clarification, learns that the oldest message is required, and then obtains the current timestamp; milestone m_0 settles with score 1.0.
- The next milestone, retrieving the target message, stays ready but never improves: its best stage score is 0 for eight consecutive observations.
- DynSTEER therefore emits a frontier-stagnation stop at replay observation 43 and maps that decision to the source execution boundary at batch 22.
- Replay accounting: the complete source trajectory records 29 execution batches and 41.04s of execution latency; the matched prefix records 22 batches and 24.68s. Stopping there would save 7 batches and 16.36s of execution latency. These are paired counterfactual savings, not a separately executed live interruption.

D.2 Case 2: First-Error Localization vs. Fluke Success

Scenario ID:

add_reminder_content_and_date_and_time_10_distraction_tools

Agent Model: DeepSeek-V4-Flash.

Execution Trace Summary:

- The agent was tasked with setting a reminder with a specific date and time.
- In intermediate steps, the agent called an invalid distraction API `set_calendar_alarm` with malformed argument types, which failed.
- The agent then stumbled upon `add_reminder` after multiple trial-and-error retries.
- Native Default Evaluation: Scored the final state as 1.0000 for full resolution, completely ignoring the 4 erroneous intermediate API calls and high token overhead.
- DynSTEER Stage Review: In the DeepSeek-V4-Flash run, the stage reaching m_0 scores 0.897, whereas the subsequent target-message stage falls to 0.272 and triggers the low-score policy. DynSTEER therefore preserves the distinction between clean execution and an unsuccessful, inefficient attempt.

E Prompt Templates for Milestone Generation and Stage Judges

This section documents the primary prompt templates used across DynSTEER’s pipeline, formatted in structured boxed environments.

Prompt Template 1: Milestone Graph Candidate Generation Prompt

System Prompt:

You generate candidate milestone goal graphs, not step-by-step trajectories. A node belongs in one candidate graph only when, under that candidate’s complete and feasible interpretation, every valid completion must reach that operation or goal. One candidate is a hypothesis about necessity; necessity across candidates is decided later by code aggregation.

User Prompt Input:

This is candidate batch {batch_index}. Focus code: {focus_code}.

Batch focus: {focus_instruction}

Core Requirements:

- Return standalone candidate graphs for the current task based strictly on the available task inputs {task_view}.
- Each graph must define its own milestones, dependency edges, dynamic bindings, and fatal minefields.
- Seek genuine diversity: explore distinct valid tools, direct state lookups, or alternative precedence orders.
- Output must strictly follow the JSON schema specifying nodes, edges, dependencies, and minefield operations.

Prompt Template 2: Standard Judge Unified Multi-Field Prompt

System Role:

You are a strict task-trajectory evaluator. Evaluate only the supplied task, stage interval, constraint checks, and trajectory steps. Do not invent external facts.

Context Inputs:

- stage_goal: Success condition for the current dominator-bounded stage interval.
- rubrics: Target dimensions to score simultaneously (e.g., correctness, efficiency, safety, tool compliance).
- steps: Observed actions and tool returns strictly within the stage evidence window $\mathcal{W}(m^*)$.

Evaluation Instructions:

- Determine whether this stage satisfies stage_goal within the supplied interval.
- Score each target dimension from 0.0 to 1.0 based on explicit behavioral evidence.
- Provide concise diagnostic justifications citing specific step indices and tool call names.
- Output format: Strict JSON mapping dimension names to score, confidence, and diagnostic rationale.

Prompt Template 3: Expensive Judge Specialist Prompt (Tool Quality Dimension)

Specialist Objective:

You are the specialist LLM-Judge for the tool_quality dimension. Score only tool-use quality.

Detailed Forensic Focus:

- Whether the agent selected the optimal tools for the active stage goal.
- Whether tool arguments are complete, accurate, and type-conforming without hallucinations.
- Whether return values, exceptions, and empty results were correctly parsed and handled rather than ignored.
- Penalize redundant retry loops, irrelevant exploratory calls, and unhandled tool exceptions.
- Output: Calibrated continuous score in $[0, 1]$, confidence score, and forensic audit trace.